

Testing Research Software

October 6, 2026

Contents

About this guide	1
Normative language and the ladder	2
Terminology	2
At a glance	3
1 Where correctness comes from	3
1.1 A test is only as good as its oracle	3
1.2 Know the answer before you ask	4
1.3 Test invariants, not only examples	5
1.4 Change the input, predict the output	5
1.5 Plant the answer, then find it	5
1.6 Check the order of convergence	6
1.7 Get a second opinion	6
2 Writing tests	7
2.1 Compare floats with tolerance	7
2.2 Flaky tests cry wolf	7
2.3 Most bugs live at the edges	7
2.4 Let the computer pick the inputs	7
2.5 Many small tests, few big ones	8
2.6 Every test stands on its own	8
2.7 Unchanged is not correct	8
3 Keeping tests honest	9
3.1 Every bug becomes a test	9
3.2 A test that cannot fail tests nothing	9
3.3 Skipped is not passed	9
3.4 Tests are code too	9

About this guide

This guide is about testing research software: showing that the code computes what it claims to compute. Together with *Design Principles for Research Software*, cited below as *Design Principles*, it forms one set of guidelines for all code written for a research purpose, in academia or in industry, in any programming language and scientific field. It is meant for looking things up while working, not for reading from cover to cover.

A test runs code on an input and compares the result with an expected value. The comparison is easy; knowing the expected value is hard. In research software, the typical failure is not a crash but a plausible, wrong number, and a test catches it only if its expected value is right and comes from somewhere other than the code under test. A test is therefore only as good as its oracle, the source of

its expected value. Section 1 is about where oracles come from, Section 2 about writing tests, and Section 3 about keeping a test suite honest as the code changes.

Tests show that the thing is built right: that the code computes what the method defines. Whether it is the right thing, that is, whether the method fits the scientific question, is validation, a scientific question that no test can answer (*Design Principles*, “Build the right thing before building the thing right”).

Two related topics belong to *Design Principles*. That code must be tested, and which code, is stated in its principles “Testing is not optional” and “Test the whole workflow on a small dataset”. Checks that the code makes on every run, such as input validation and assertions of invariants, are not tests; they are the subject of “Garbage in, useful error out” and “Wear belt and braces”.

Normative language and the ladder

The words *must*, *should*, and *may*, and the plain imperatives “prefer” and “avoid”, carry the meaning defined in the subsection “Normative language” of *Design Principles*. The rules of this guide apply along the same “Ladder of Research Software”: the *musts* are part of the floor from the second rung on, and what rises with the rung is how many kinds of oracle the tests use, how strong they are, and how much of the input space they reach. When rules conflict, the priorities in *Design Principles*, “Get your priorities right” decide. Each rule is stated in one of the two documents only.

Terminology

The terms defined in *Design Principles*, such as package, library code, workflow code, entry point, host stack, and caller, carry the same meaning here. The terms of testing, such as test, test suite, oracle, and tolerance, and the types of tests, such as unit test, end-to-end test, and snapshot test, are defined in *Types of Research Software Tests*, which also describes what each type catches and what it misses. This guide uses them with that meaning.

At a glance

Every principle of this guide in one line. The number and the title link to the principle.

1 Where correctness comes from

1.1	A test is only as good as its oracle	Never take an expected value from the code under test.
1.2	Know the answer before you ask	Use exact results from analytic solutions, hand calculations, and certified references, and record their source.
1.3	Test invariants, not only examples	Check properties that every correct result has, such as conservation laws, bounds, and normalizations.
1.4	Change the input, predict the output	Check how the output must change when the input changes in a known way.
1.5	Plant the answer, then find it	Simulate data with known parameters, and check that the method recovers them with honest uncertainty.
1.6	Check the order of convergence	Check that the error of a discretization falls at the rate of the method.
1.7	Get a second opinion	Compare with an independent implementation, ideally a naive one written for the test.

2 Writing tests

2.1	Compare floats with tolerance	Compare within a tolerance derived from the numerical error of the method, not bit for bit.
2.2	Flaky tests cry wolf	Fix the random generator, and assert statistical claims with bounds that hold for any seed.
2.3	Most bugs live at the edges	Test empty, single, tied, missing, and extreme inputs, and the documented behavior there.
2.4	Let the computer pick the inputs	Check invariants and relations on generated inputs, the same on every run of the suite.
2.5	Many small tests, few big ones	Write mostly fast unit tests, a few end-to-end tests, and run slow tests separately.
2.6	Every test stands on its own	No dependence on other tests, global state, or the network; no confidential data.
2.7	Unchanged is not correct	Store only validated outputs, compare them within a tolerance, and explain every update.

3 Keeping tests honest

3.1	Every bug becomes a test	Reproduce each bug in a failing test before fixing it.
3.2	A test that cannot fail tests nothing	See every new test fail, for example by breaking the code on purpose.
3.3	Skipped is not passed	Report every skipped test with its reason, and never silence a failing one.
3.4	Tests are code too	Keep tests simple, name each for its claim, and make failures explain themselves.

1 Where correctness comes from

1.1 A test is only as good as its oracle

A test compares a result with an expected value, and it is only as good as the source of that value, its oracle. A test meant to show that a result is correct must not take its expected value from the code under test. Running the code and copying its output into the test makes the test pass by construction; it is then a snapshot test (Subsection 2.7), which shows that the result has not changed since, not that it is right.

An oracle can be wrong too, and a test that agrees with a wrong oracle is worse than no test, because it lends the wrong result authority. Prefer oracles that are simpler than the code under test and can be checked by reading them, and state where each comes from (Subsection 1.2).

No single kind of oracle catches every error. A known answer checks a few points of the input space exactly; an invariant checks every input, but only one property of the result; parameter recovery checks the whole method on realistic data, but only within sampling error. Each entry point should be tested against at least one oracle that is independent of its code, and the more depends on a method, the more kinds of oracle should test it. Table 1 lists strong oracles for common kinds of code.

Table 1: Strong oracles for common kinds of code. The numbers refer to the subsections of this guide.

Code	Typical silent error	Strong oracles
Numerical routines, such as transforms, special functions, and solvers of linear systems	A wrong result in part of the input range, such as for large magnitudes or nearly singular input	Known answers (1.2), invariants (1.3), a naive implementation (1.7)
Discretizations, such as solvers of differential equations and quadrature rules	Convergence at a lower order than the method has	The order of convergence and manufactured solutions (1.6), conservation laws (1.3)
Statistical estimators and model fits	Biased estimates, or standard errors and intervals that are too narrow	Parameter recovery with coverage (1.5), certified reference datasets (1.2), an independent implementation (1.7)
Stochastic simulations	The right mean with the wrong distribution	Statistical bounds on distributions (2.2), limiting cases with known answers (1.2), invariants (1.3)
Data transformations, such as parsing, joining, normalizing, and filtering	Rows dropped, duplicated, or misaligned with their identifiers	Tiny cases checked by hand (1.2), reordering and duplication (1.4), counts (1.3)
Workflows	Steps that do not fit together, or options that never reach their step	An end-to-end run on simulated input with a known result (1.5), counts and totals (1.3)

1.2 Know the answer before you ask

The most direct oracle is a known answer: the exact result for a particular input, obtained without the code under test. Sources include:

- Analytic solutions and limiting cases, such as a model whose closed form is known for one value of a parameter, or the behavior of an estimator as the sample size grows.
- Tiny cases worked out by hand or with a computer algebra system, such as the likelihood of three observations or the alignment of two short sequences.
- Certified reference values, such as the Statistical Reference Datasets of NIST, whose results are certified to many digits.
- Published values from the literature or from another established tool.

State next to each expected value where it comes from: the derivation, the hand calculation, or the reference, with the table and the number of digits given. An expected value without a stated source cannot be checked by a reviewer; it is a magic number (*Design Principles*, “No magic numbers”).

Make sure that the reference computes the same quantity. Published values often rest on other conventions or defaults: a variance divided by n or by $n - 1$, a rate or a scale parameterization, another rule for ties, or another convergence tolerance (*Design Principles*, “Numerics and defaults are part of the API”). A disagreement may then be a difference of conventions rather than a bug.

Derive the tolerance from the precision of the reference: a value published with four significant digits cannot test the fifth.

Known answers are exact but sparse. They exist mostly for small or simple inputs and so test the easy corner of the input space; the other kinds of oracle in this section reach the rest.

1.3 Test invariants, not only examples

Passing on a few happy-path inputs is a weak scientific test. Prefer invariants: properties that every correct result has, whatever the input. Examples are conservation laws, such as of mass, energy, or the total of the counts; normalizations, such as probabilities that sum to one; bounds, such as a nonnegative variance or a p-value between zero and one; symmetries, such as a symmetric distance matrix; and consistency relations, such as row and column totals that agree.

An invariant needs no known answer, so it can be checked on any input, including large, random, and extreme ones where no known answer exists (Subsection 2.4). It checks only one property of the result, however, and a result can have it and still be wrong: probabilities that sum to one can still be the wrong probabilities. Combine invariants with other oracles (Subsection 1.1).

Examples are useful, but they do not prove that the method is correct. Encode the scientific claims the software makes and test those claims directly. The invariants worth testing are often the ones worth asserting in the code itself (*Design Principles*, “Wear belt and braces”): the test checks them on the inputs someone chose, the assertion on the data that users bring.

1.4 Change the input, predict the output

Where the correct result for an input is unknown, its relation to the result for a related input often is known. Change the input in a way whose effect on the output the method determines, and test that the output changes accordingly. Such metamorphic relations include:

- **Reordering.** Permuting the observations leaves an estimate unchanged, and permuting the rows of the input permutes the rows of the output in the same way, with the identifiers still aligned.
- **Rescaling.** Expressing the input in other units changes the result by the corresponding factor, and shifting the coordinates shifts an estimated position.
- **Duplicating and splitting.** Duplicating every observation leaves a maximum likelihood estimate unchanged and divides its standard error by $\sqrt{2}$. Splitting the data, computing an additive statistic such as a count or a sum on each part, and combining the parts gives the result for the whole.
- **Monotonicity.** A higher confidence level never gives a narrower interval, and a higher detection threshold never detects more events.

Such relations catch errors that leave each single result plausible, such as two axes mixed up, identifiers misaligned after a sort, or a unit converted twice. Like invariants, they need no known answer and can be checked on any input (Subsection 2.4). Derive each relation from the method, not from the code. A relation that holds only approximately, for example because reordering changes the rounding of a sum or because an iterative method stops at a tolerance, is checked within a tolerance of its own (Subsection 2.1).

1.5 Plant the answer, then find it

Statistical methods, and inverse problems in general, should be tested by parameter recovery: simulate data from the model with known parameters, apply the method, and check that it recovers them. For such code it is often the strongest test, because it exercises the whole method on data of realistic size and structure, where no hand-computed answer exists.

Check the uncertainty as well as the estimate. An estimator can find the right value and still report standard errors or intervals that are too narrow, and this is the more dangerous error, because it makes wrong conclusions look certain. Over many simulated datasets, the estimates should scatter around the true values as much as the reported standard errors say, and the intervals should contain the true values at their nominal rate, their coverage. Simulate under the null hypothesis as well: a test at the 5% level should then reject in about 5% of the datasets. Include hard cases, such as small samples, parameters near the boundary of their range, and weak signals, because that is where estimators break first.

Assert recovery within bounds derived from the sampling error, with a known and very small false-failure rate (Subsection 2.2). The simulator is the oracle, so it should be written from the definition of the model, independently of the code of the method: a simulator and an estimator that share a mistake, such as the same wrong parameterization, agree perfectly. Studies of coverage need many replicates and are slow; run them with the slow tests (Subsection 2.5).

1.6 Check the order of convergence

A discretization, such as a solver of differential equations, a quadrature rule, or a finite-difference scheme, should be tested by its observed order of convergence, not only by its error at one resolution. Run it at a sequence of resolutions against an exact solution, estimate the order from how fast the error falls, and compare it with the order the method has: halving the step of a second-order method divides the error by about four. Choose resolutions fine enough that the leading term of the error dominates and coarse enough that rounding error does not.

The error at one resolution is a weak test. A bug that reduces a second-order method to first order, such as a boundary condition applied one step late, still gives a small error on a fine enough grid, and a tolerance loose enough to pass at one resolution passes such bugs too. The order is a sharp oracle: a single number known from the theory of the method, which most bugs of a discretization change.

Where no exact solution exists, manufacture one. Choose a smooth function as the solution, insert it into the equations to obtain the source terms and boundary conditions it requires, add these to the problem, and test that the solver converges to the chosen function at the expected order. The same idea applies beyond grids: the error of a Monte Carlo estimate falls as $N^{-1/2}$ in the number of samples, and an iterative solver converges at the rate its theory gives.

1.7 Get a second opinion

Compare results with an independent implementation of the same method. The best is often a naive one, written for the test: a plain loop instead of vectorized code, a dense matrix instead of a sparse one, a direct sum instead of a fast transform, or all permutations instead of a random sample of them. It is slow, but simple enough to be checked by reading (*Design Principles*, “KISS: Keep it simple, stupid”), and on small inputs it provides an expected value for every input the fast code accepts. Keep it in the test code, not in the package.

Another established package, possibly in another language, gives a second opinion as well. Either call it in the test, with its version pinned in the test environment, or store its results as known answers together with its version (Subsection 1.2); a new release of it can change its numbers (*Design Principles*, “Numerics and defaults are part of the API”). Compare the same quantity under the same conventions.

Agreement is evidence only as far as the implementations are independent. Two implementations written from the same derivation, from the same paper, or by the same person share their misreadings and agree on the wrong answer. A disagreement does not say which implementation is wrong; resolve it with another oracle before changing either.

2 Writing tests

2.1 Compare floats with tolerance

Tests should compare floating-point results within a tolerance, not bit for bit. Bit-identical reference files treat platform noise as a logic bug and make the test suite fragile.

Assert agreement within the tolerance that defines reproducibility (*Design Principles*, “Once is luck, twice is science”). Use relative tolerances for quantities of varying magnitude and absolute tolerances for quantities near zero. Tolerances are constants in the sense of *Design Principles*, “No magic numbers”: derive each from the expected numerical error of the method, rather than adjusting it until the tests pass on one machine. A tolerance that is too loose is as harmful as one that is too tight, because it passes wrong results; it should be smaller than the smallest error that would matter (Subsection 3.2).

2.2 Flaky tests cry wolf

A test that fails at random teaches people to ignore failures, and a test suite whose failures are ignored checks nothing. Tests of stochastic methods must therefore fix the random generator (*Design Principles*, “Once is luck, twice is science”), so that every run gives the same outcome.

A fixed generator makes a test repeatable, not convincing: a test that passes for only one seed can hide a bug. Where the claim is statistical, such as an unbiased estimate or a coverage rate, assert it with bounds whose false-failure rate is known and very small, for example a deviation of several standard errors, so that the test would pass for other seeds as well. The bound is a constant in the sense of *Design Principles*, “No magic numbers”. Never choose a seed because the test passes with it.

Randomness is not the only cause of flaky tests. Tests that depend on timing, on the network, on the order in which tests run, or on parallel reductions that sum in varying order fail at random as well (Subsection 2.6).

2.3 Most bugs live at the edges

The tests of every entry point should cover the edges of its input space, not only typical inputs, because code breaks at the edges and typical test data never go there. Edges include:

- **Sizes.** Empty input, a single element, a single group or category, and sizes that do not divide evenly into chunks or batches.
- **Values.** Zeros, ties, duplicates, columns with zero variance, and values at the boundary of a parameter’s range, such as a probability of zero or one.
- **Missing and special values.** NA, NaN, infinities, and masked entries.
- **Magnitudes.** Very small and very large values, and nearly singular or ill-conditioned inputs (*Design Principles*, “Prefer numerically stable formulations”).
- **Structure.** Unsorted input, and identifiers that look like numbers or contain spaces.

At each edge, test the documented behavior: a correct result, or the documented error or warning (*Design Principles*, “Raise, warn, or stay silent”). The policy for missing values of each entry point (*Design Principles*, “Mind the gap, don’t fill it”) is tested this way, with missing values in every position that the policy treats differently. Where nothing is documented for an edge, writing the test reveals the gap in the documentation.

2.4 Let the computer pick the inputs

People choose test inputs that they expect to work. Where an invariant (Subsection 1.3) or a metamorphic relation (Subsection 1.4) holds for a whole domain of inputs, test it on generated inputs

as well as on chosen ones. Property-based testing frameworks, such as Hypothesis for Python and quickcheck for R, generate hundreds of inputs per test, search the edges deliberately (Subsection 2.3), and reduce a failing input to a minimal one, which makes the bug easy to see.

Generate inputs from the whole domain that the function documents. A domain narrowed until the tests pass excludes exactly the inputs the test was meant to find.

Generated inputs are random, so Subsection 2.2 applies: in the test suite, they must be the same on every run. Search for new failing inputs separately, for example with the slow tests (Subsection 2.5), and turn every failing input the search finds into a fixed test (Subsection 3.1).

2.5 Many small tests, few big ones

Most tests should be unit tests: each runs one function on a tiny input and finishes in a fraction of a second. A suite of such tests runs in minutes, so it is run before every change rather than avoided, and a failing unit test points to the function that broke. A few end-to-end tests then run whole entry points on a small dataset, to show that the parts fit together; for workflow code, such a test is required (*Design Principles*, “Test the whole workflow on a small dataset”).

Tests that are slow by nature, such as studies of coverage (Subsection 1.5), convergence on fine grids (Subsection 1.6), or comparisons on large inputs, should be marked as slow and run separately, on a schedule and before every release, rather than dropped. Slowness is no reason to skip a test silently (Subsection 3.3). Performance at the scale of real applications is the subject of benchmarks, not of tests (*Design Principles*, “Size matters”).

2.6 Every test stands on its own

Each test should set up everything it needs and leave nothing behind, so that it gives the same outcome alone, in any order, and in parallel with other tests. A test that relies on a seed, an option, or a file left by another test passes or fails depending on which tests ran before it, and a test that changes global state, such as the random generator, the working directory, or display options, breaks others (*Design Principles*, “Do not touch global state”). Write files only to a temporary directory that the test runner provides and removes.

Tests should depend on nothing outside the repository and the declared test dependencies, such as the network, a database, a particular machine, or the current date. Such dependencies make tests fail for reasons unrelated to the code (Subsection 2.2), and the outcome of a test run cannot be reproduced later.

Keep test data small and in the repository. Prefer synthetic data generated in the test from a fixed seed, which documents how the data were made and keeps them next to the claim they test. Where real data matter, for example for a file format, use an excerpt small enough for the repository and licensed for redistribution. Confidential data, such as patient records, must not be part of a test suite: a repository is copied to every machine and every person that works on the code, and its history keeps every file ever committed.

2.7 Unchanged is not correct

A snapshot test compares a result with a reference output stored from an earlier run of the code. It catches silent changes, such as numbers that move after a refactoring or an update of a dependency, and it is often the only practical test of a large output. But it shows only that the result is unchanged, not that it is correct: a snapshot of a wrong result protects the error.

Store a reference output only after checking it against an oracle (Section 1), and record how it was made: the version of the package and the parameters (*Design Principles*, “Make every result traceable to a version”). Compare it within a tolerance (Subsection 2.1), not bit for bit.

A failing snapshot test means that a result changed, and someone has to decide whether the change is a fix, an intended change of numerical behavior, or a bug. The reference output must not be regenerated just to make the suite pass. Each update should state why the result changed, and a change of numerical behavior goes into the changelog (*Design Principles*, “Design stable APIs”).

3 Keeping tests honest

3.1 Every bug becomes a test

Every fixed bug should gain a test that fails without the fix. Write the test first: reproduce the bug, see the test fail, then fix the code and see the test pass. A test written after the fix may pass for reasons unrelated to the bug, and only its failure shows that it detects the bug.

A bug is also evidence about the tests. Ask which oracle would have caught it and why none did, and look for the same mistake elsewhere in the code: a wrong index or a wrong convention is rarely made only once.

3.2 A test that cannot fail tests nothing

Every new test should be seen to fail at least once. A test that has only ever passed may never have checked anything. Common reasons are an assertion that is never reached, for example inside a loop over an empty list; a result compared with itself, or with an expected value from the same code (Subsection 1.1); a tolerance so loose that any result passes; an error caught and discarded inside the test; and a test that the test runner never collects, for example because its file is misnamed.

Break the code on purpose and check that a test fails: flip a sign, drop the last element, swap two axes, or change a constant by a few percent. Mutation testing tools, available for several host stacks, make such changes automatically and report those that no test detected.

Code coverage, the share of the code that the tests run, shows what is not tested, not what is. Code that no test runs is untested, but code that a test runs is tested only as far as the test checks its result.

3.3 Skipped is not passed

A test that does not run checks nothing, and a test report that hides it looks like success (*Design Principles*, “Better loud than wrong”). Tests that are skipped, for example because an optional dependency or an accelerator is missing, and tests marked as expected failures must therefore appear as such in the test report, with their reason.

Every test should run in at least one configuration of the automatic test runs. A test that is skipped everywhere leaves its code path unexercised, and such a path is not supported (*Design Principles*, “Testing is not optional”): make the test run, or stop claiming support. A failing test must not be deleted, disabled, or given a looser tolerance just to make the suite pass. A known bug that cannot be fixed yet may be marked as an expected failure that refers to its bug report, so that the suite shows when it is fixed.

3.4 Tests are code too

Test code follows the same principles as the code it tests: it is read, reviewed, and maintained, and it is read most carefully when a test fails. Keep each test simple, with as little logic, branching, and looping as possible; a test complex enough to need tests of its own cannot be trusted to check anything. Define shared knowledge once, such as tolerances and the generation of test data (*Design Principles*, “DRY: Don’t repeat yourself”).

Each test should check one claim, and its name should state that claim, such as that an estimate does not depend on the order of the rows, rather than a number. When a test fails, its message should

show the expected value, the result, and the tolerance, so that the failure can be understood without running the test again.